

Step 1: Importing necessary libraries

```
In [1]: import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from wordcloud import WordCloud
import warnings
warnings.filterwarnings(action = "ignore")
```

Step 2: Importing the dataset

```
In [2]: data = pd.read_csv("Amazon_Cell_Phones.csv", encoding = "unicode_escape")
```

Step 3: Sanity check of data

```
In [3]: # to get first two rows of data
data.shape
```

```
Out[3]: (3351, 19)
```

```
In [4]: data.head(2)
```

```
Out[4]:
```

	ID	product_name	product_link	image_link	(D
0	1	Apple iPhone XR, 64GB, Black - Unlocked (Renewed)	https://www.amazon.com/Apple-iPhone-XR-Fully-U...	https://m.media-amazon.com/images/I/717KHGCJ6e...	1
1	2	Apple iPhone 11, 64GB, Black - Unlocked (Renewed)	https://www.amazon.com/Apple-iPhone-11-64GB-Bl...	https://m.media-amazon.com/images/I/612kg3rGyY...	2

```
In [5]: data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3351 entries, 0 to 3350
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   ID                    3351 non-null   int64
 1   product_name          3351 non-null   object
 2   product_link          3351 non-null   object
 3   image_link            3351 non-null   object
 4   Price (Dollar)        2347 non-null   float64
 5   discount_percentage   3351 non-null   float64
 6   price_before_discount 569 non-null    float64
 7   rating_out_of_5       3117 non-null   float64
 8   number_of_ratings     3117 non-null   float64
 9   brand                 3347 non-null   object
10   operating_system      2947 non-null   object
11   RAM (GB)              1894 non-null   float64
12   CPU                   1222 non-null   object
13   Storage (GB)          3256 non-null   float64
14   screen_size (Inches)  3163 non-null   float64
15   cellular_technology    2810 non-null   object
16   model_name            3185 non-null   object
17   cpu_model             576 non-null    object
18   available_colors      1287 non-null   object
dtypes: float64(8), int64(1), object(10)
memory usage: 497.5+ KB

```

```

In [6]: # missing values
data.isnull().sum()

```

```

Out[6]: ID                    0
product_name                 0
product_link                 0
image_link                   0
Price (Dollar)              1004
discount_percentage          0
price_before_discount       2782
rating_out_of_5             234
number_of_ratings           234
brand                       4
operating_system            404
RAM (GB)                    1457
CPU                          2129
Storage (GB)                 95
screen_size (Inches)        188
cellular_technology         541
model_name                   166
cpu_model                    2775
available_colors             2064
dtype: int64

```

```

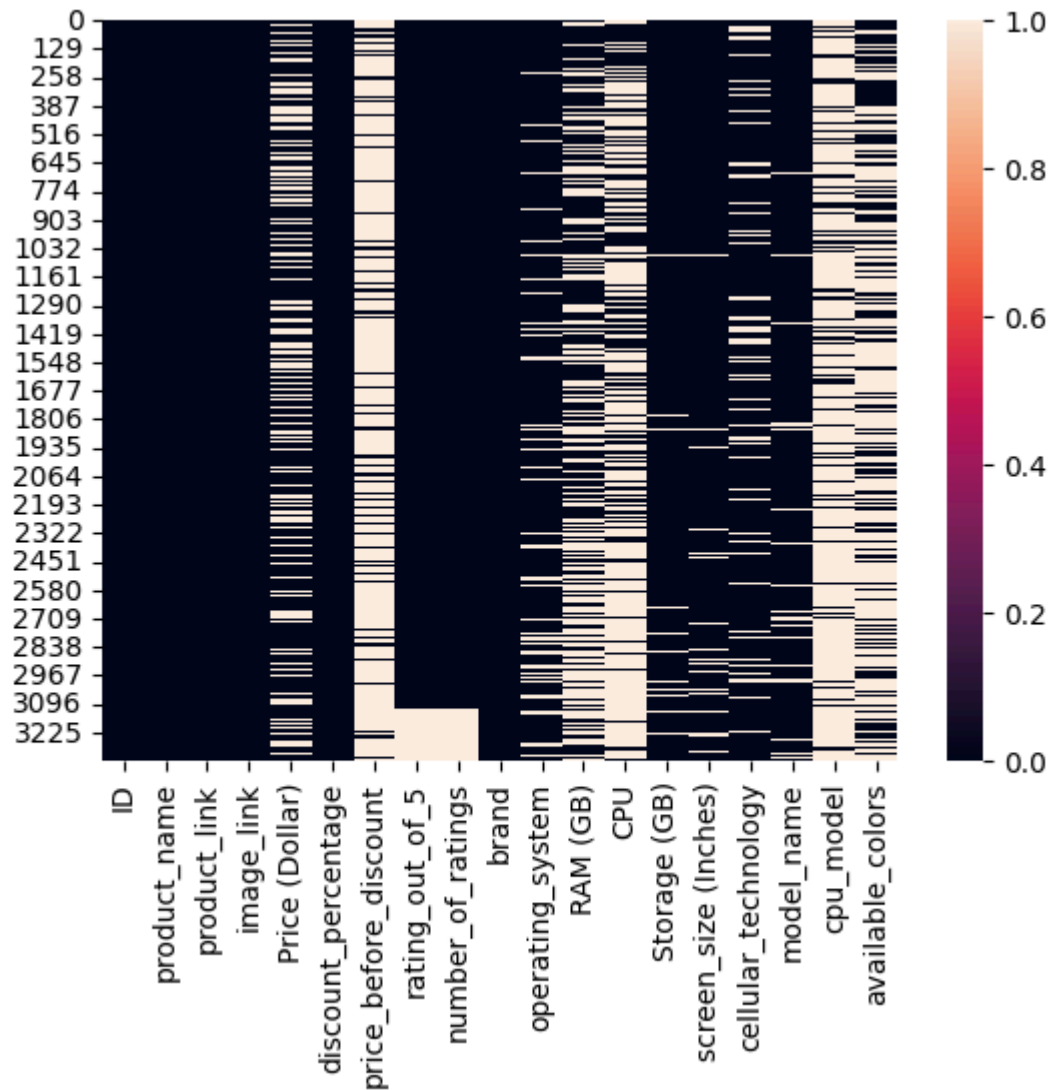
In [7]: # Graphical Representation
sns.heatmap(data.isnull())

```

```

Out[7]: <Axes: >

```



Step 4: Data Cleaning and Preprocessing

```
In [8]: # Percentage of missing values
data.isnull().sum()/data.shape[0]
```

```
Out[8]: ID                0.000000
        product_name      0.000000
        product_link      0.000000
        image_link        0.000000
        Price (Dollar)    0.299612
        discount_percentage 0.000000
        price_before_discount 0.830200
        rating_out_of_5   0.069830
        number_of_ratings 0.069830
        brand             0.001194
        operating_system   0.120561
        RAM (GB)           0.434796
        CPU                0.635333
        Storage (GB)       0.028350
        screen_size (Inches) 0.056103
        cellular_technology 0.161444
        model_name         0.049537
        cpu_model          0.828111
        available_colors   0.615936
        dtype: float64
```

```
In [9]: # Overall percent of missing values
        (data.isnull().sum().sum()/(data.shape[0] * data.shape[1])) * 100
```

```
Out[9]: 22.109660902480012
```

```
In [10]: # Let's treat the missing values for numerical data
data['Price (Dollar)'].fillna(data['Price (Dollar)'].mean(), inplace=True)
data['RAM (GB)'].fillna(data['RAM (GB)'].median(), inplace=True)
data['screen_size (Inches)'].fillna(data['screen_size (Inches)'].mean(), inplace=True)
data['rating_out_of_5'].fillna(data['rating_out_of_5'].mean(), inplace=True)

# For categorical columns
data['brand'].fillna(data['brand'].mode()[0], inplace=True)
data['operating_system'].fillna(data['operating_system'].mode()[0], inplace=True)
data['Storage (GB)'].fillna(data['Storage (GB)'].mode()[0], inplace=True)

# Filling CPU with Unknown
data['CPU'].fillna('Unknown', inplace=True)
data['available_colors'].fillna('Unknown', inplace=True)
data['cellular_technology'].fillna('Unknown', inplace=True)
data['model_name'].fillna('Unknown', inplace=True)
data['cpu_model'].fillna('Unknown', inplace=True)
data['price_before_discount'].fillna('unknown', inplace=True)

# Filling with 0
data['number_of_ratings'].fillna(0, inplace=True)
```

Step 5: EDA

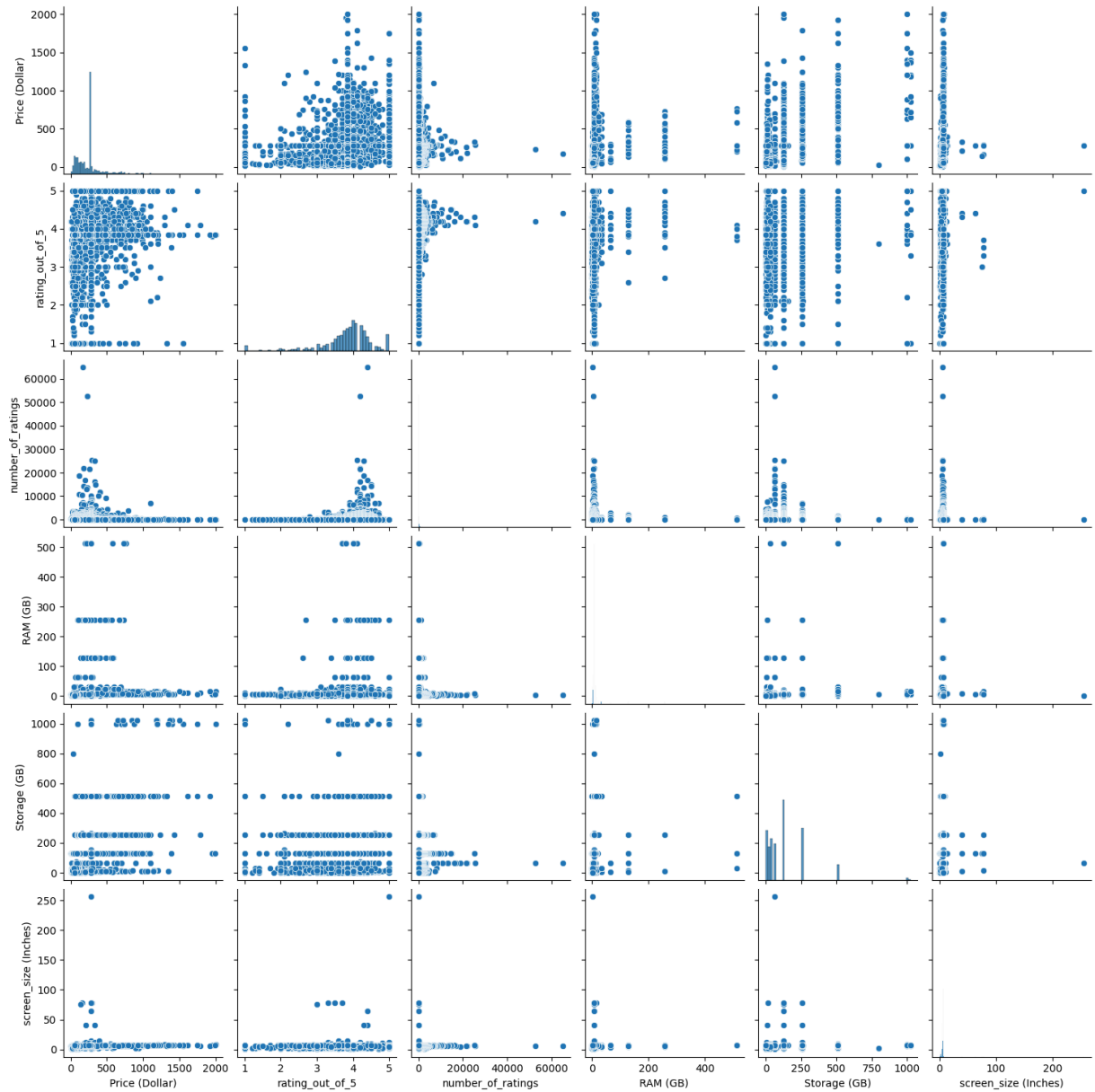
```
In [11]: # descriptive_statistics
data.describe(include = "all").T
```

Out[11]:

	count	unique	top	freq	mean
ID	3351.0	NaN	NaN	NaN	1676.0
product_name	3351	3351	Apple iPhone XR, 64GB, Black - Unlocked (Renewed)	1	NaN
product_link	3351	3351	https://www.amazon.com/Apple-iPhone-XR-Fully-U...	1	NaN
image_link	3351	2968	https://m.media-amazon.com/images/I/61afOyd+zE...	10	NaN
Price (Dollar)	3351.0	NaN	NaN	NaN	278.421172
discount_percentage	3351.0	NaN	NaN	NaN	3.47678
price_before_discount	3351	380	unknown	2782	NaN
rating_out_of_5	3351.0	NaN	NaN	NaN	3.837632
number_of_ratings	3351.0	NaN	NaN	NaN	373.840645
brand	3351	280	SAMSUNG	1002	NaN
operating_system	3351	207	Android	1066	NaN
RAM (GB)	3351.0	NaN	NaN	NaN	9.990394
CPU	3351	151	Unknown	2129	NaN
Storage (GB)	3351.0	NaN	NaN	NaN	125.659239
screen_size (Inches)	3351.0	NaN	NaN	NaN	6.058097
cellular_technology	3351	130	4G	948	NaN
model_name	3351	1707	Unknown	166	NaN
cpu_model	3351	66	Unknown	2778	NaN
available_colors	3351	800	Unknown	2064	NaN

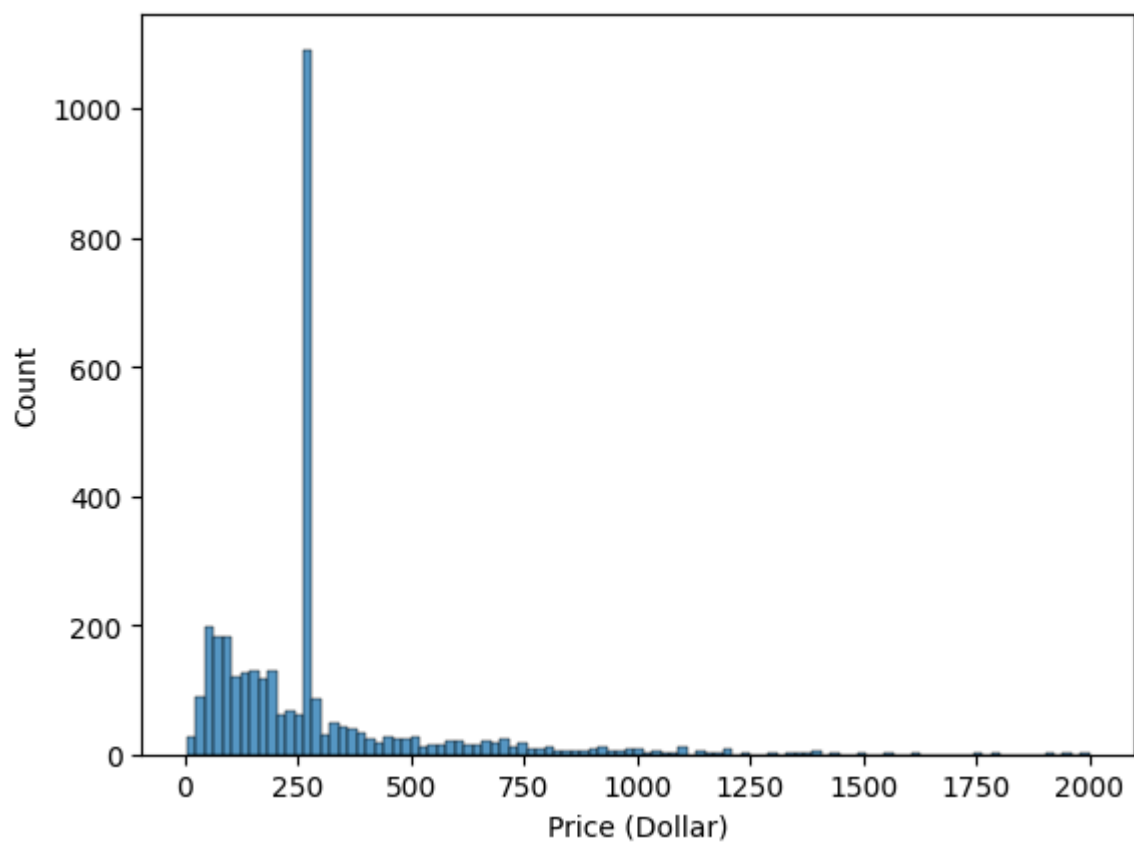
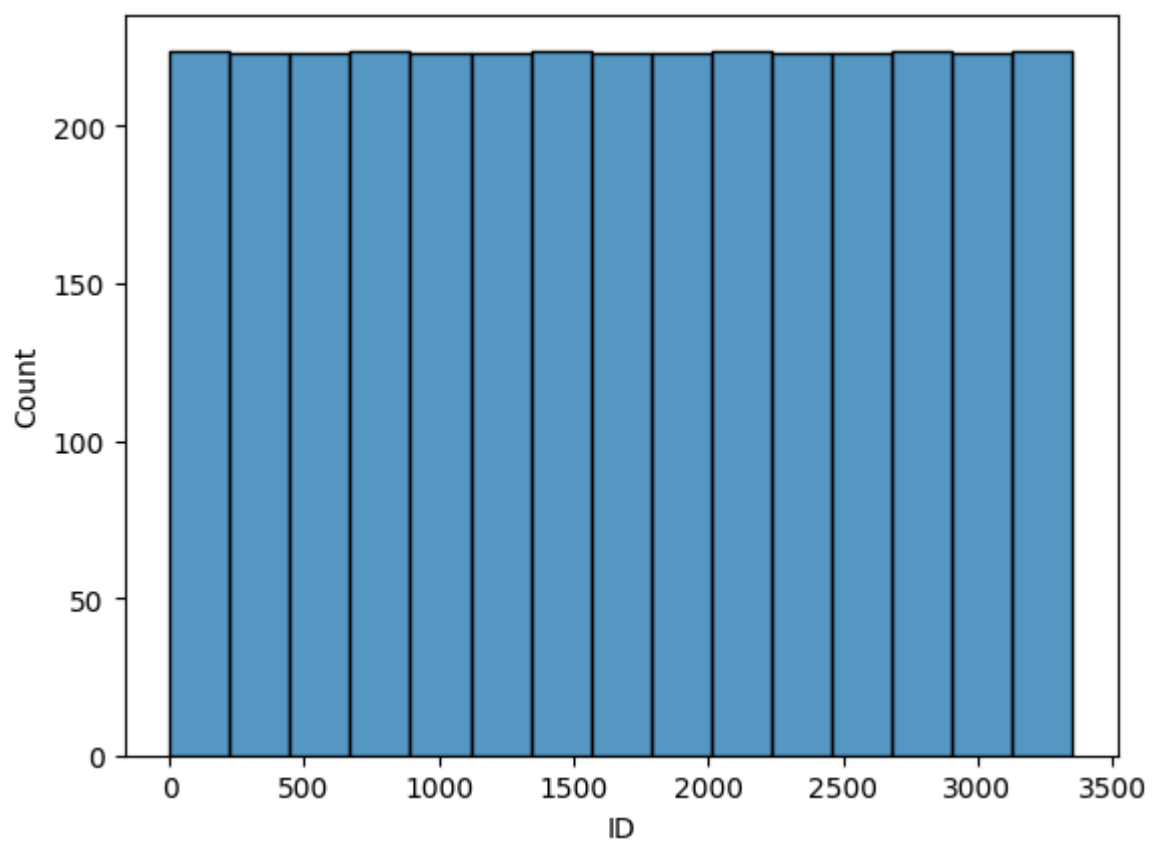
1. Visualize the pairplot of data for numerical columns

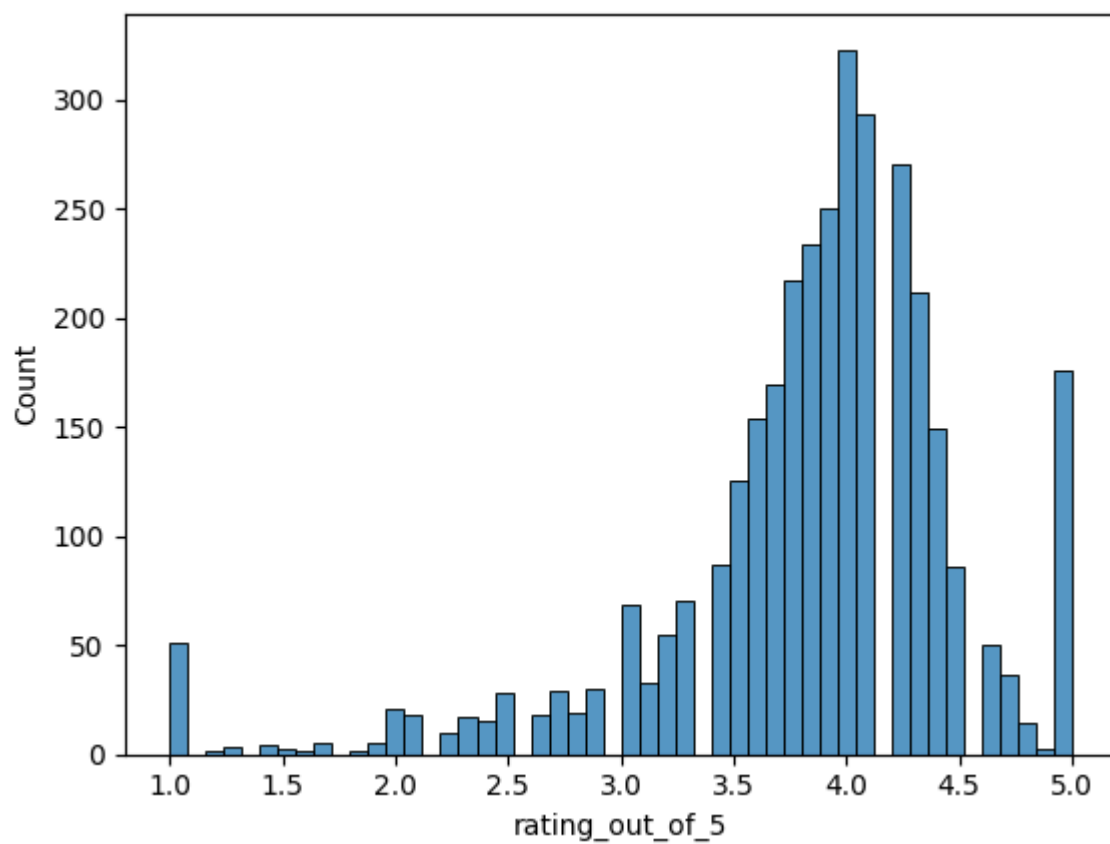
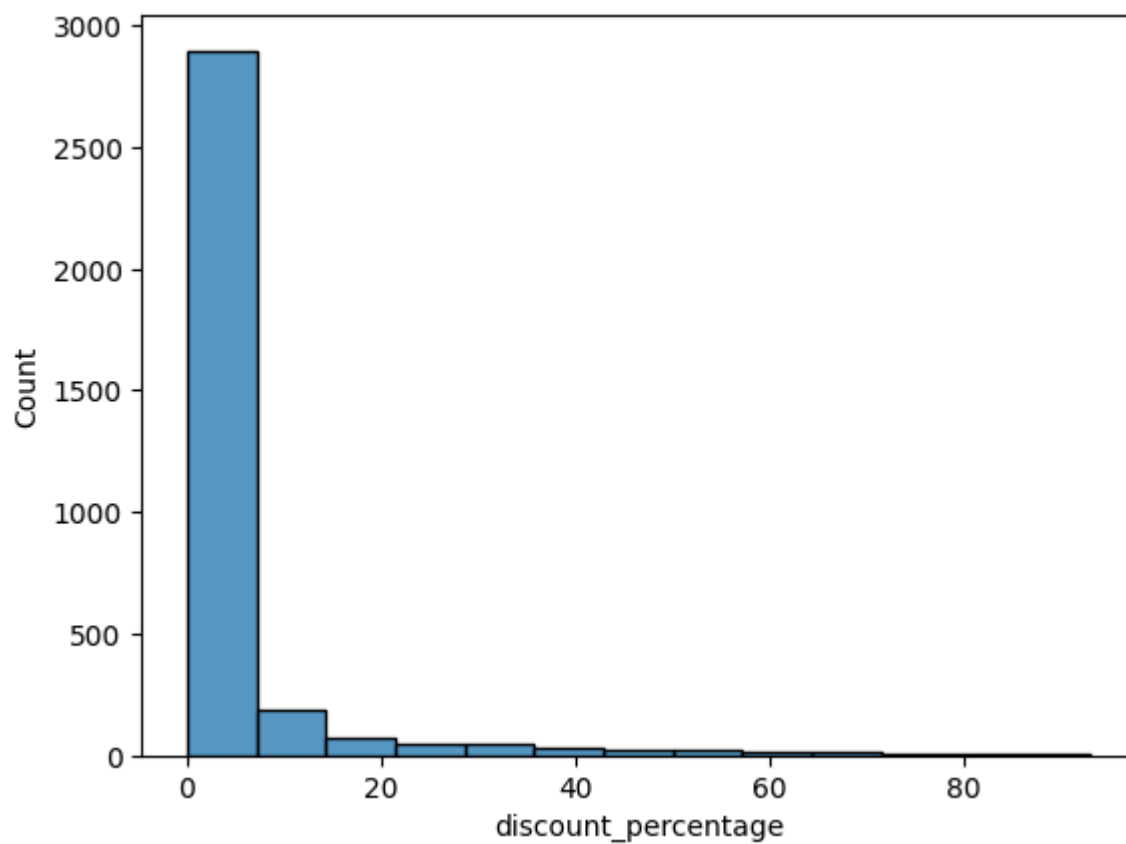
```
In [12]: data_numeric = data[['Price (Dollar)', 'rating_out_of_5', 'number_of_ratings', 'RAM
sns.pairplot(data_numeric)
plt.show()
```

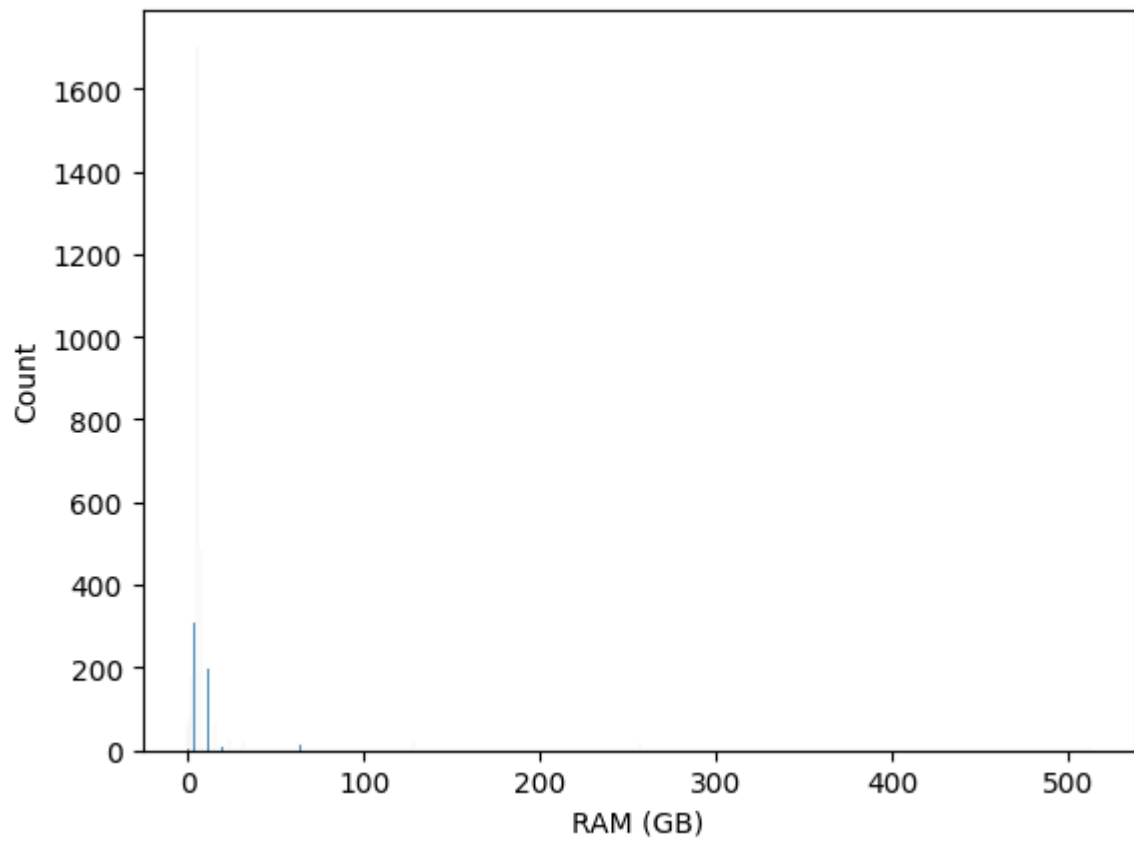
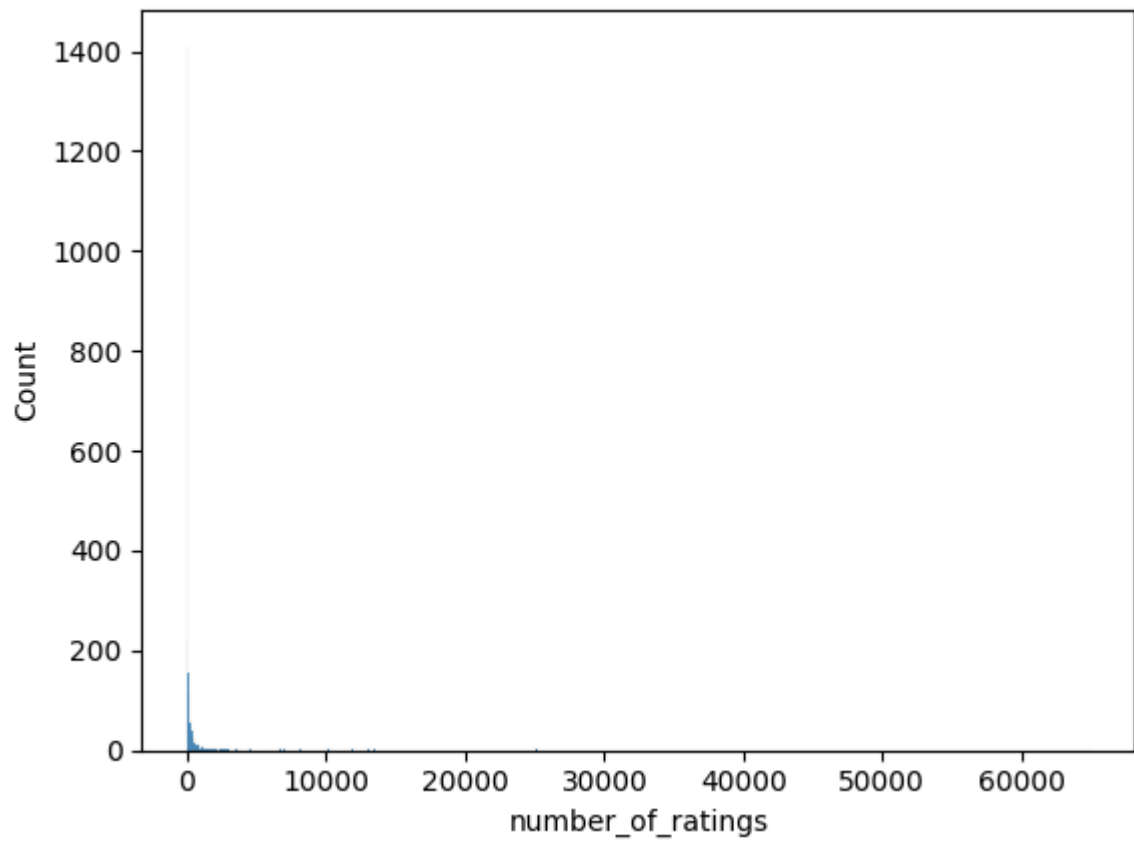


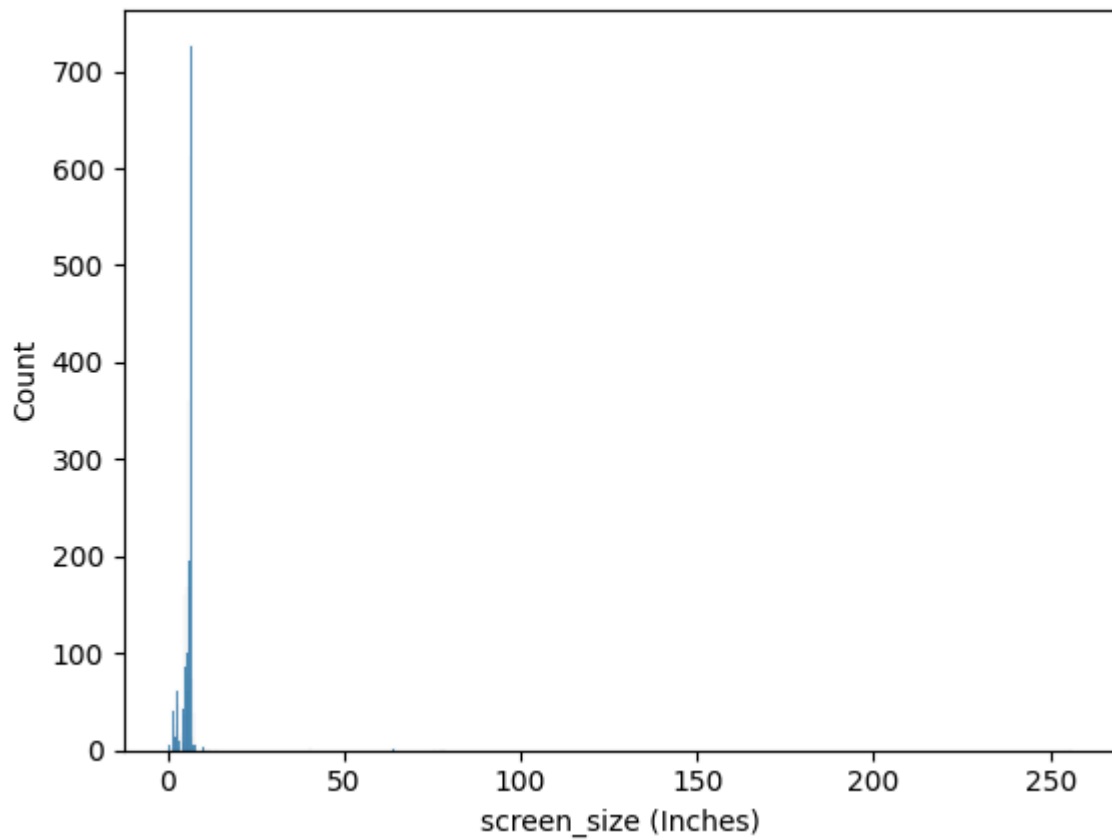
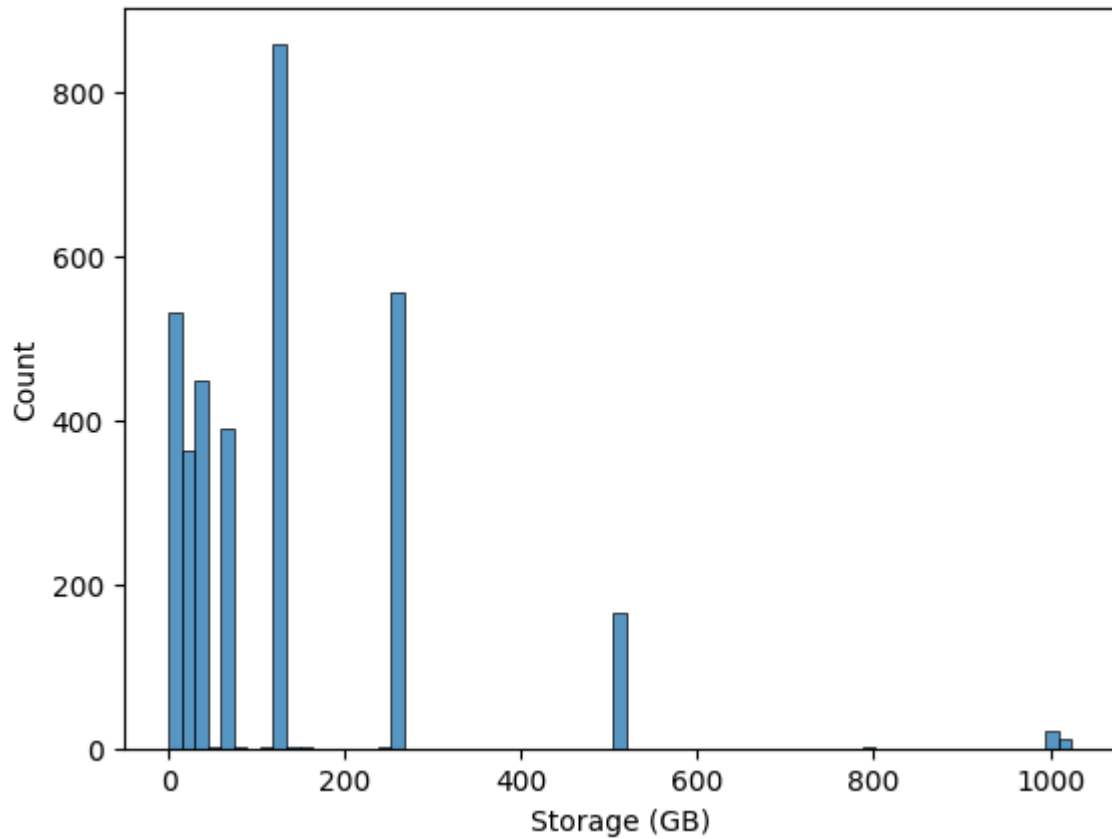
2. Histogram to understand the distribution

```
In [13]: for i in data.select_dtypes(include = "number").columns:
sns.histplot(data = data, x = i)
plt.show()
```



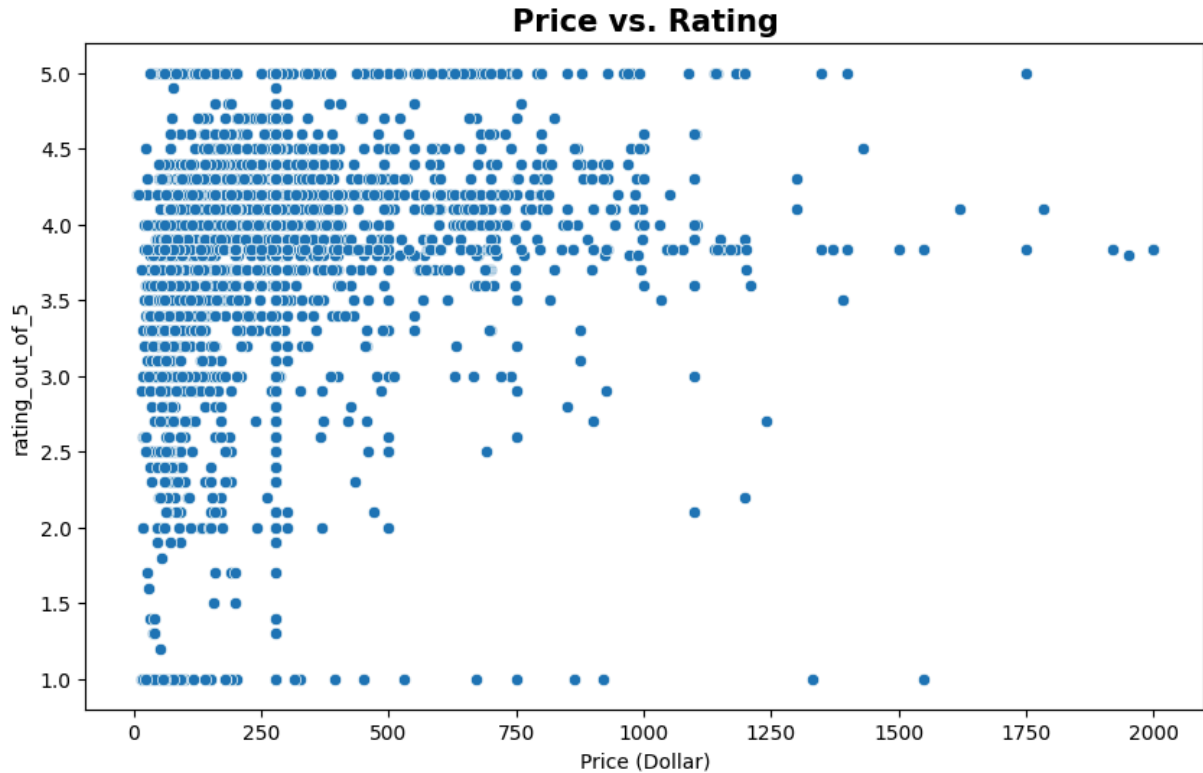






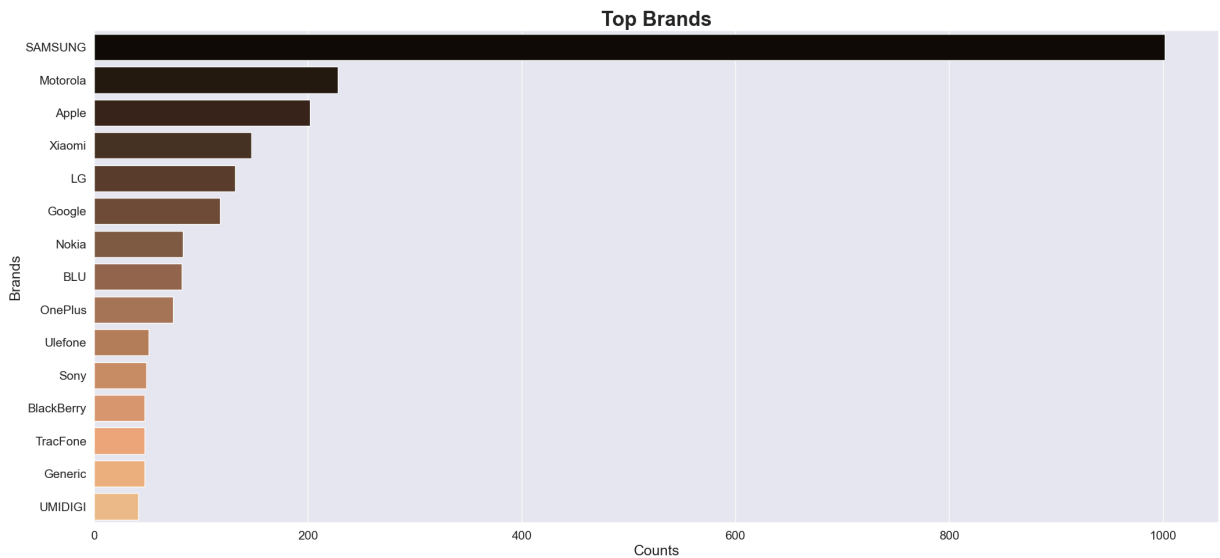
```
In [14]: # Scatter plot to see the relationship between price and rating
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Price (Dollar)', y='rating_out_of_5', data=data)
```

```
plt.title("Price vs. Rating", fontweight = "bold", fontsize = 15)
plt.show()
```



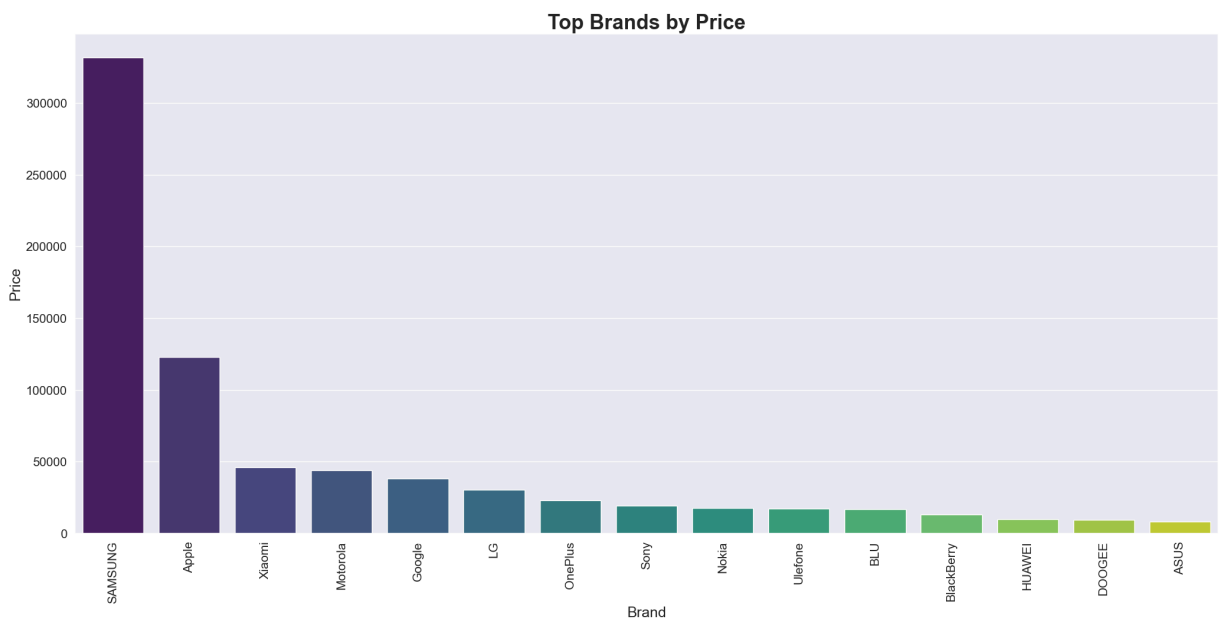
3. Top 15 Brand

```
In [15]: sns.set(rc = {'figure.figsize' : (25, 11)})
top_10_brand = data['brand'].value_counts().head(15).reset_index()
top_10_brand.columns = ['brand', 'Counts']
sns.barplot(x = "Counts", y = "brand", data = top_10_brand, palette = "copper")
plt.title("Top Brands", fontweight = "bold", fontsize = 25)
plt.xlabel("Counts", fontsize = 18)
plt.ylabel("Brands", fontsize = 18)
plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.show()
```



4. Top 15 Brand by Price

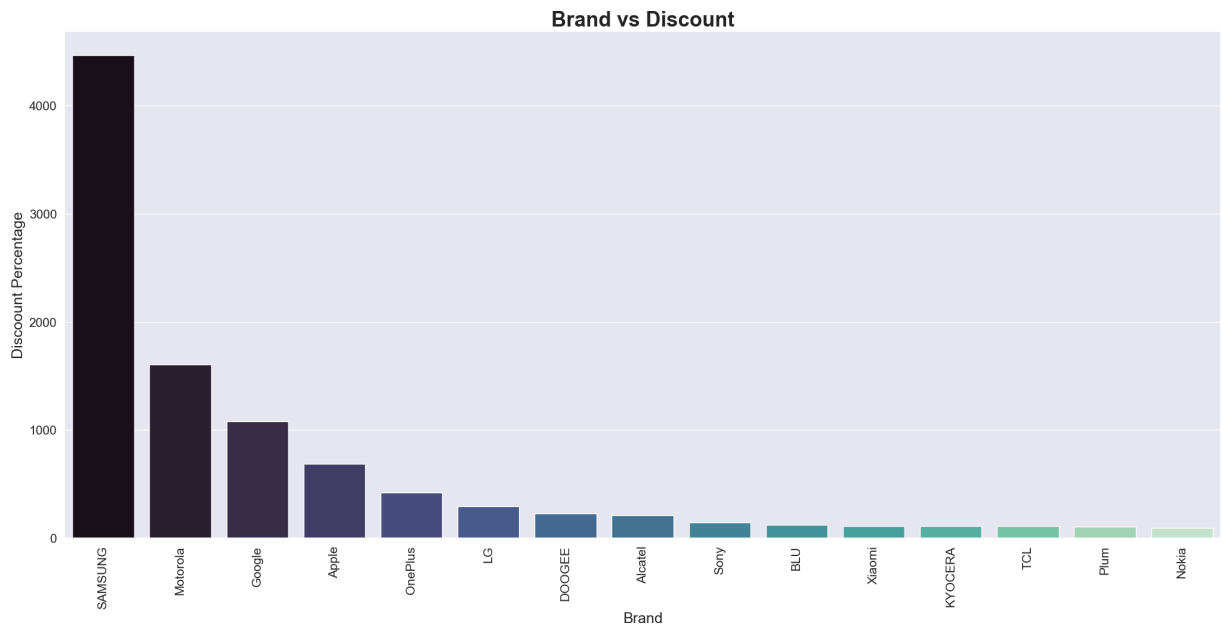
```
In [16]: brand_price = data.groupby(["brand"], as_index = False)["Price (Dollar)"].sum().sort_values(ascending=False)
sns.barplot(x = "brand", y = "Price (Dollar)", data = brand_price, palette = "virid
plt.xticks(rotation = 90)
plt.title("Top Brands by Price", fontweight = "bold", fontsize = 25)
plt.xlabel("Brand", fontsize = 18)
plt.ylabel("Price", fontsize = 18)
plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.show()
```



5. Top 15 Brand by Discount Percentage

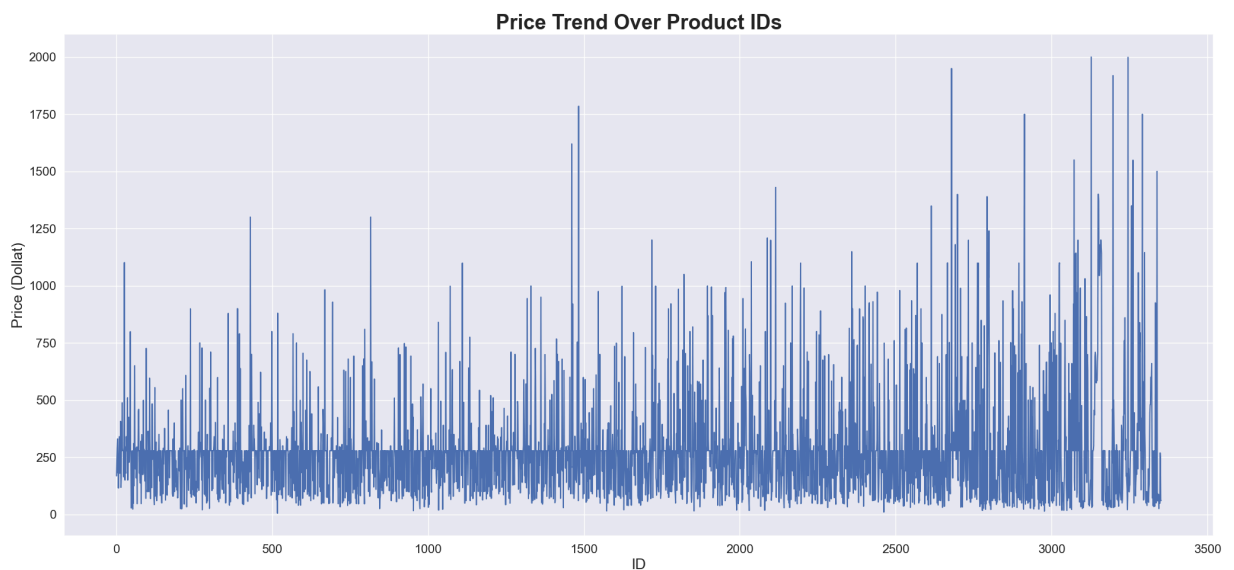
```
In [17]: brand_discount = data.groupby(["brand"], as_index = False)["discount_percentage"].sum().sort_values(ascending=False)
sns.barplot(x = "brand", y = "discount_percentage", data = brand_discount, palette = "virid
plt.xticks(rotation = 90)
```

```
plt.title("Brand vs Discount", fontweight = "bold", fontsize = 25)
plt.xlabel("Brand", fontsize = 18)
plt.ylabel("Discount Percentage", fontsize = 18)
plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.show()
```



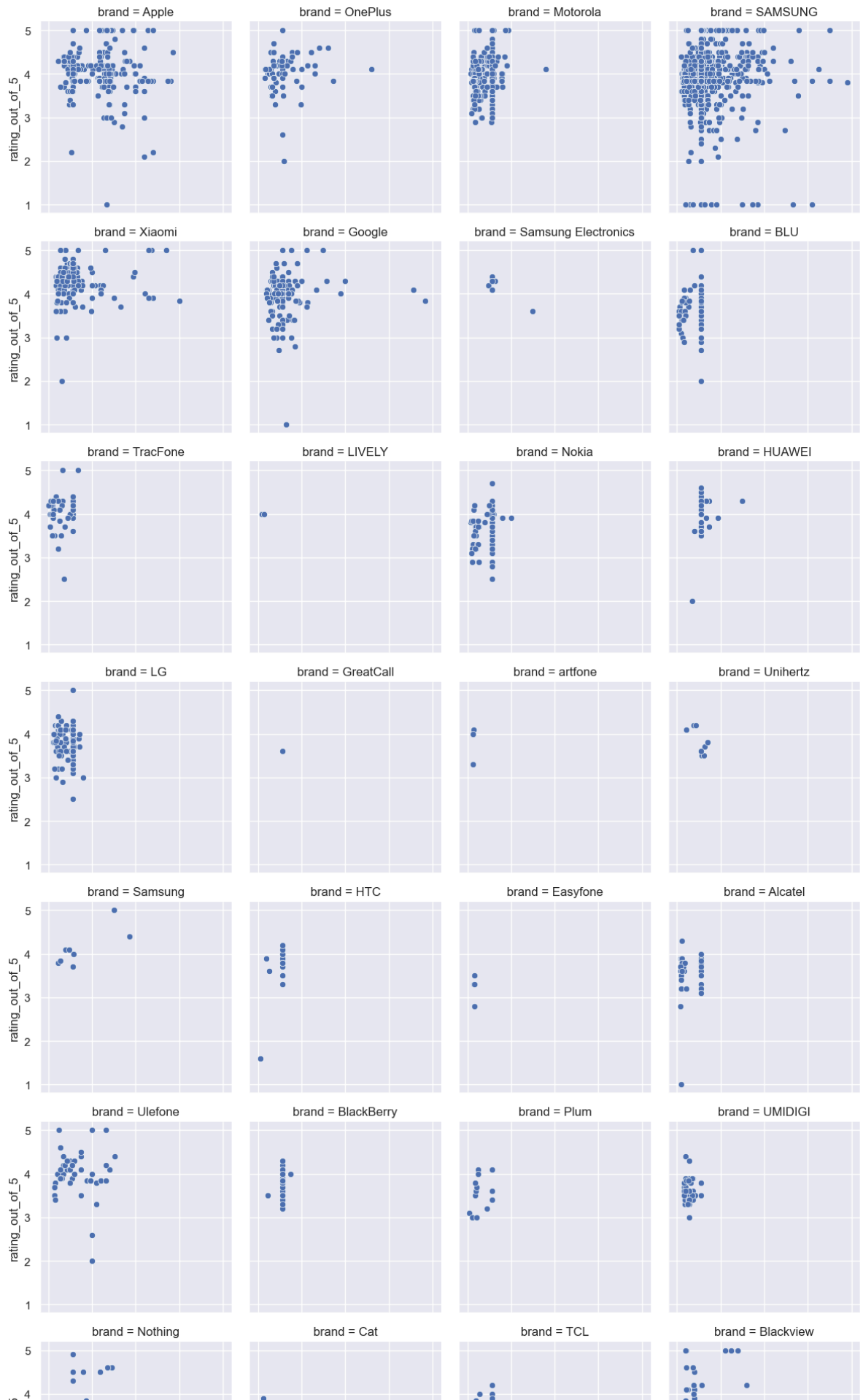
6. Price over the product ID

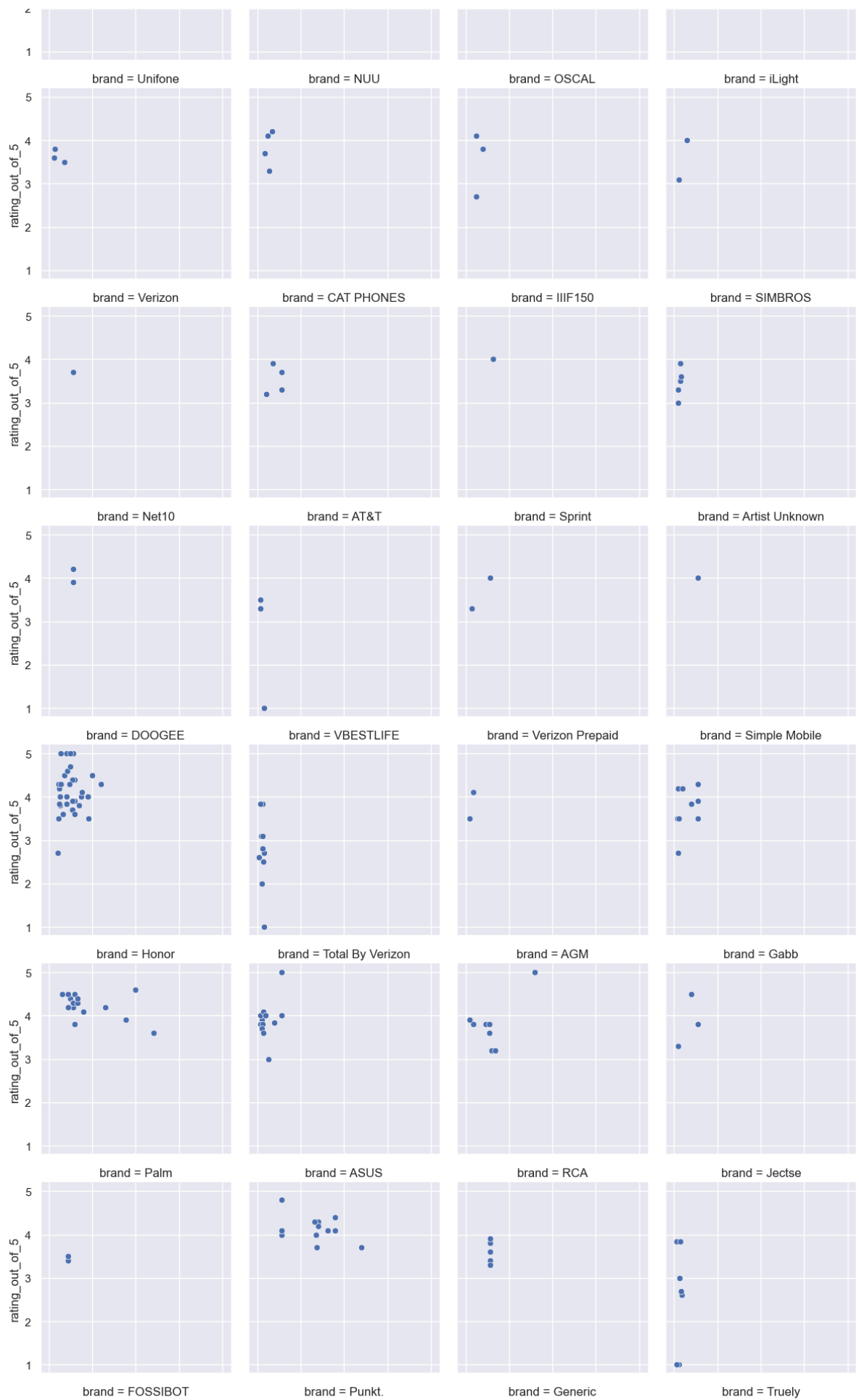
```
In [18]: sns.set(rc = {'figure.figsize' : (25, 11)})
sns.lineplot(x = "ID", y = "Price (Dollar)", data=data)
plt.title("Price Trend Over Product IDs", fontweight = "bold", fontsize = 25)
plt.xlabel("ID", fontsize = 18)
plt.ylabel("Price (Dollat)", fontsize = 18)
plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.show()
```

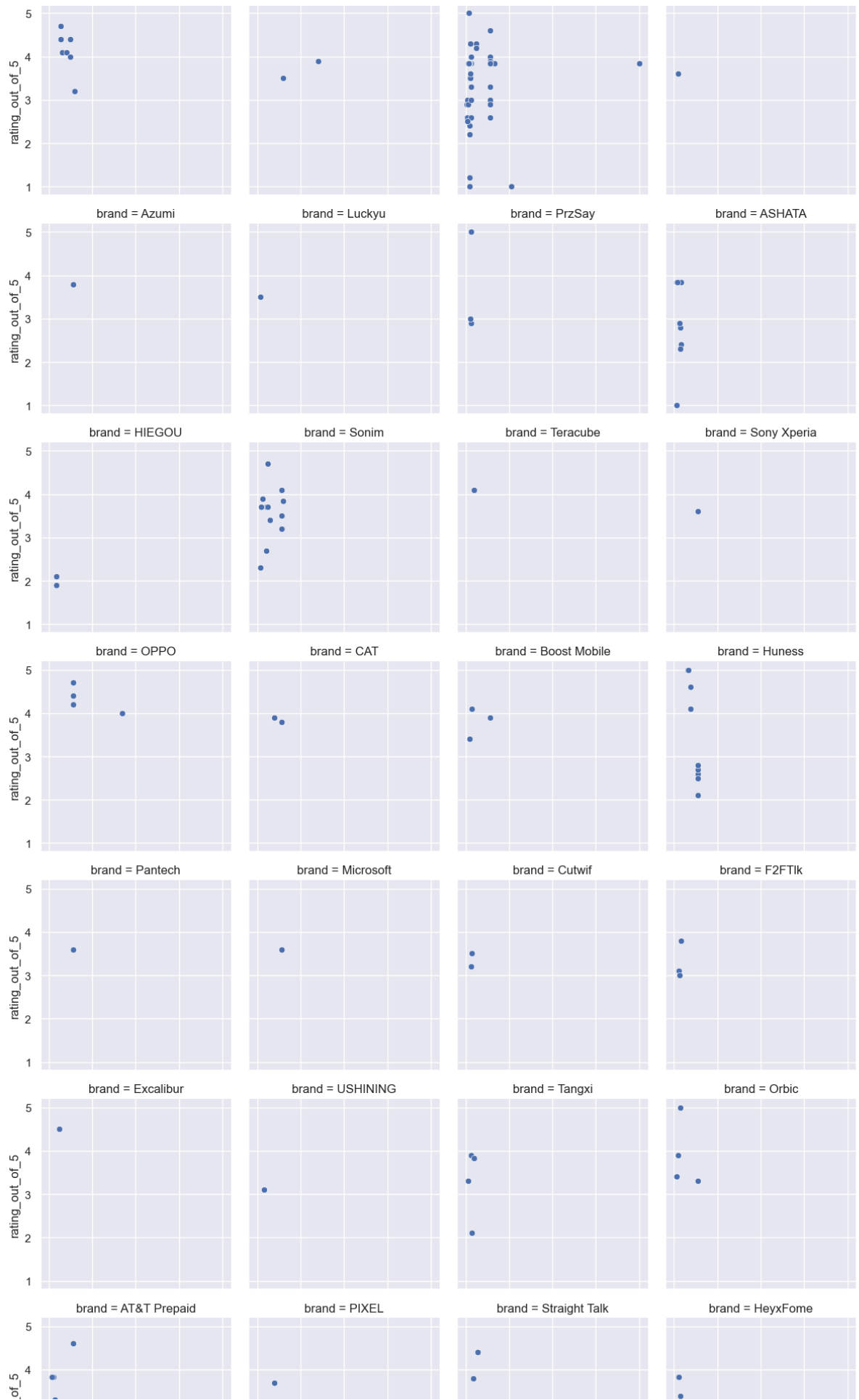


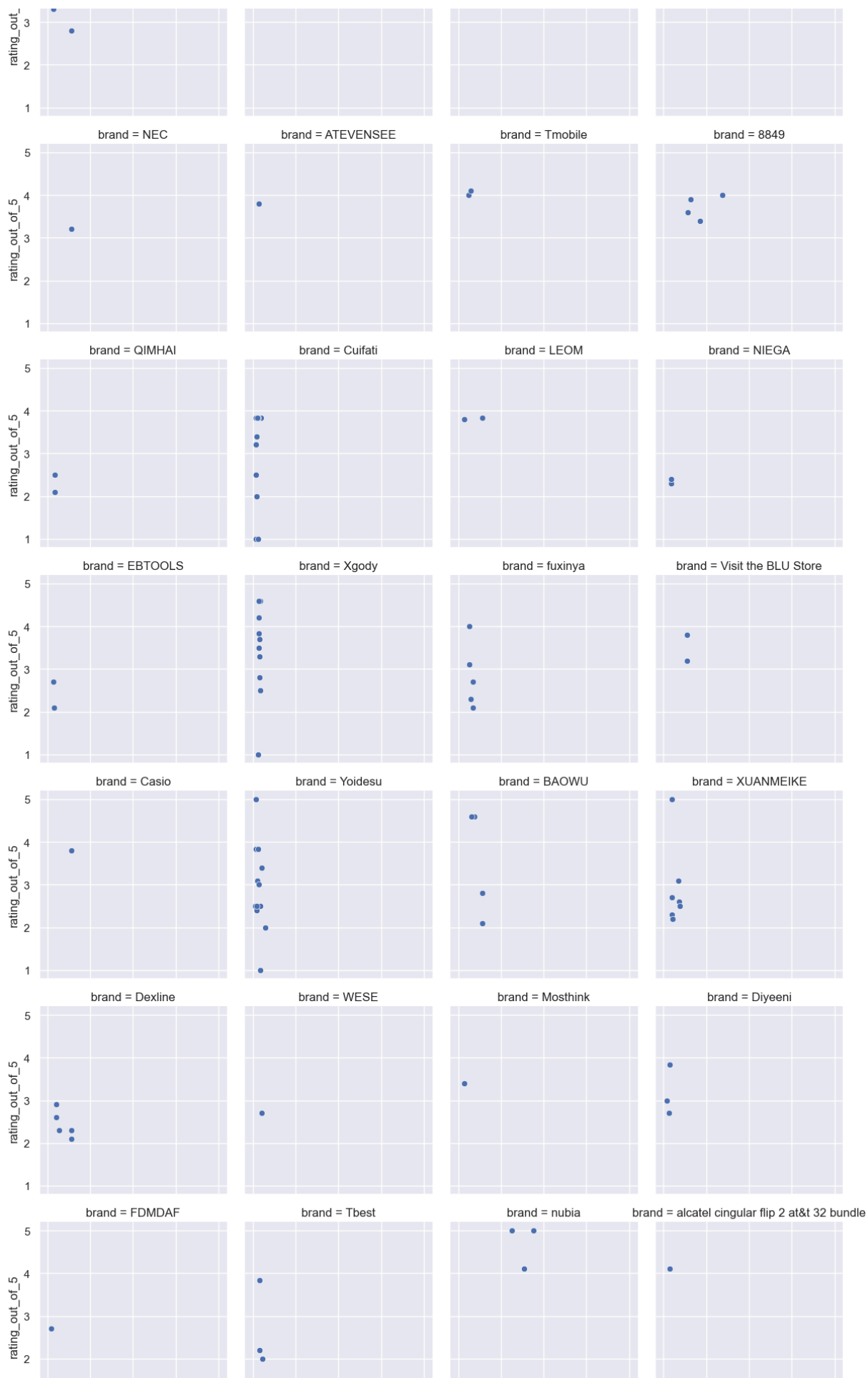
7. Price vs Rating, separated by brand

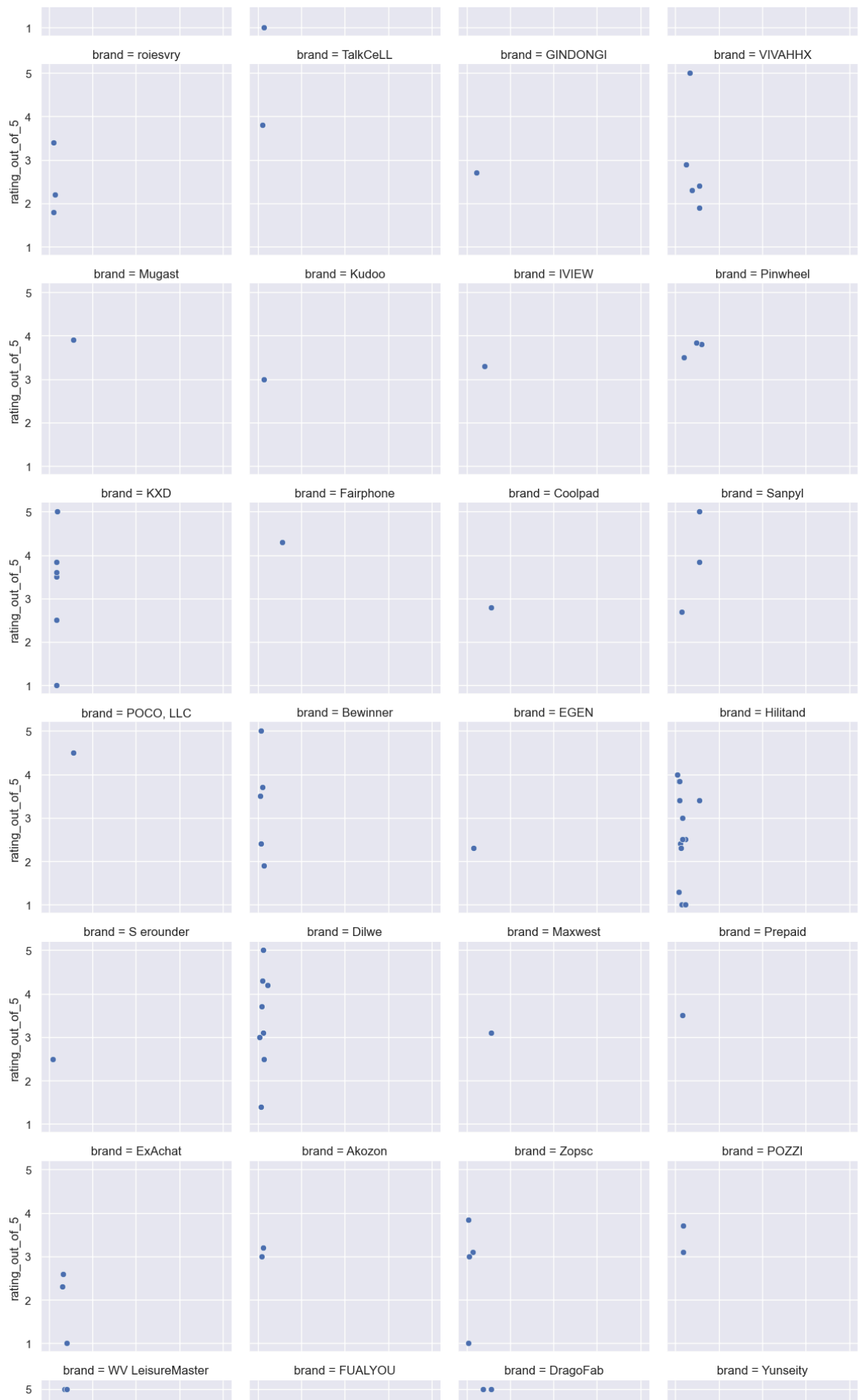
```
In [19]: g = sns.FacetGrid(data, col='brand', col_wrap=4, height=3)
g.map(sns.scatterplot, 'Price (Dollar)', 'rating_out_of_5')
plt.show()
```

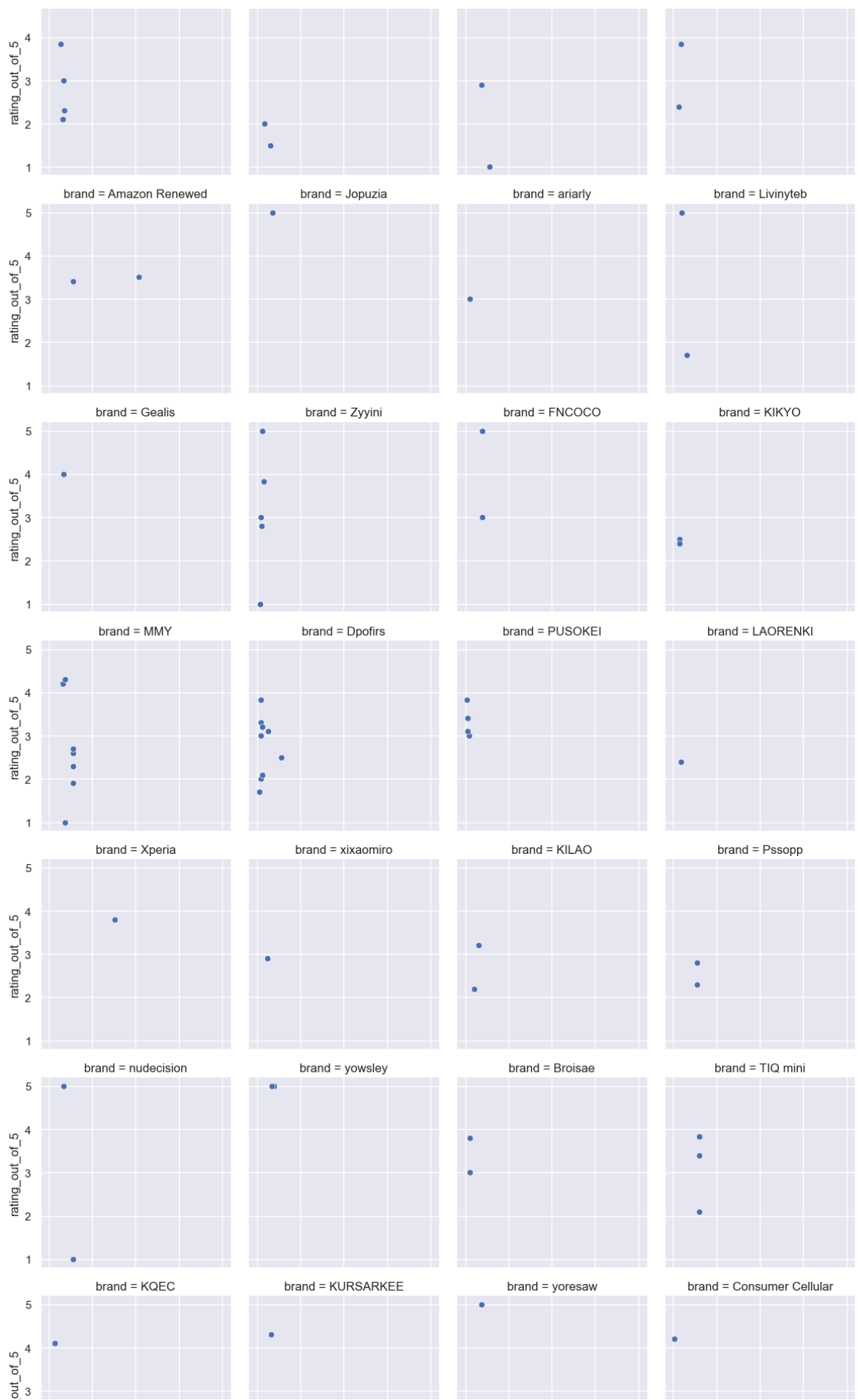


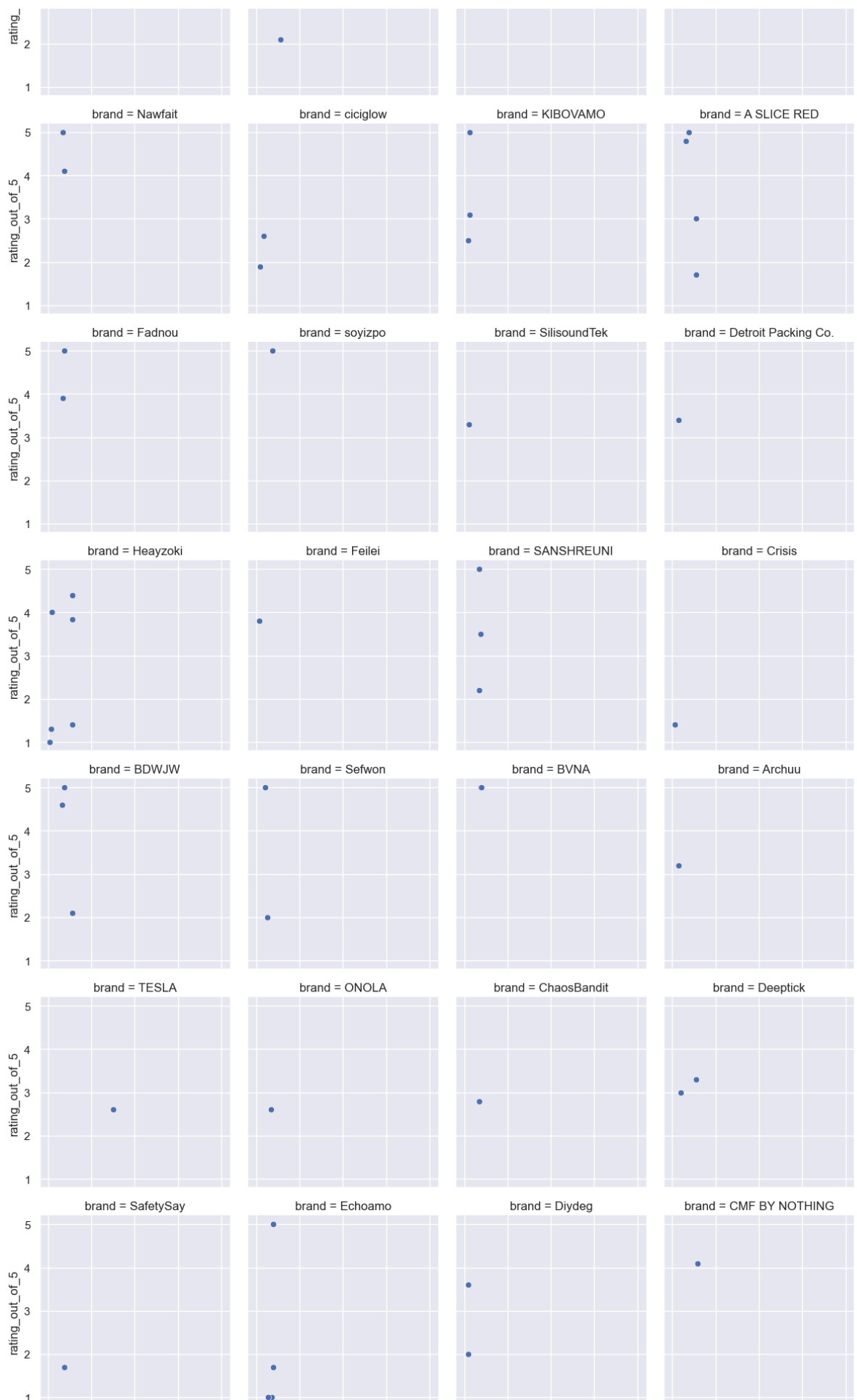


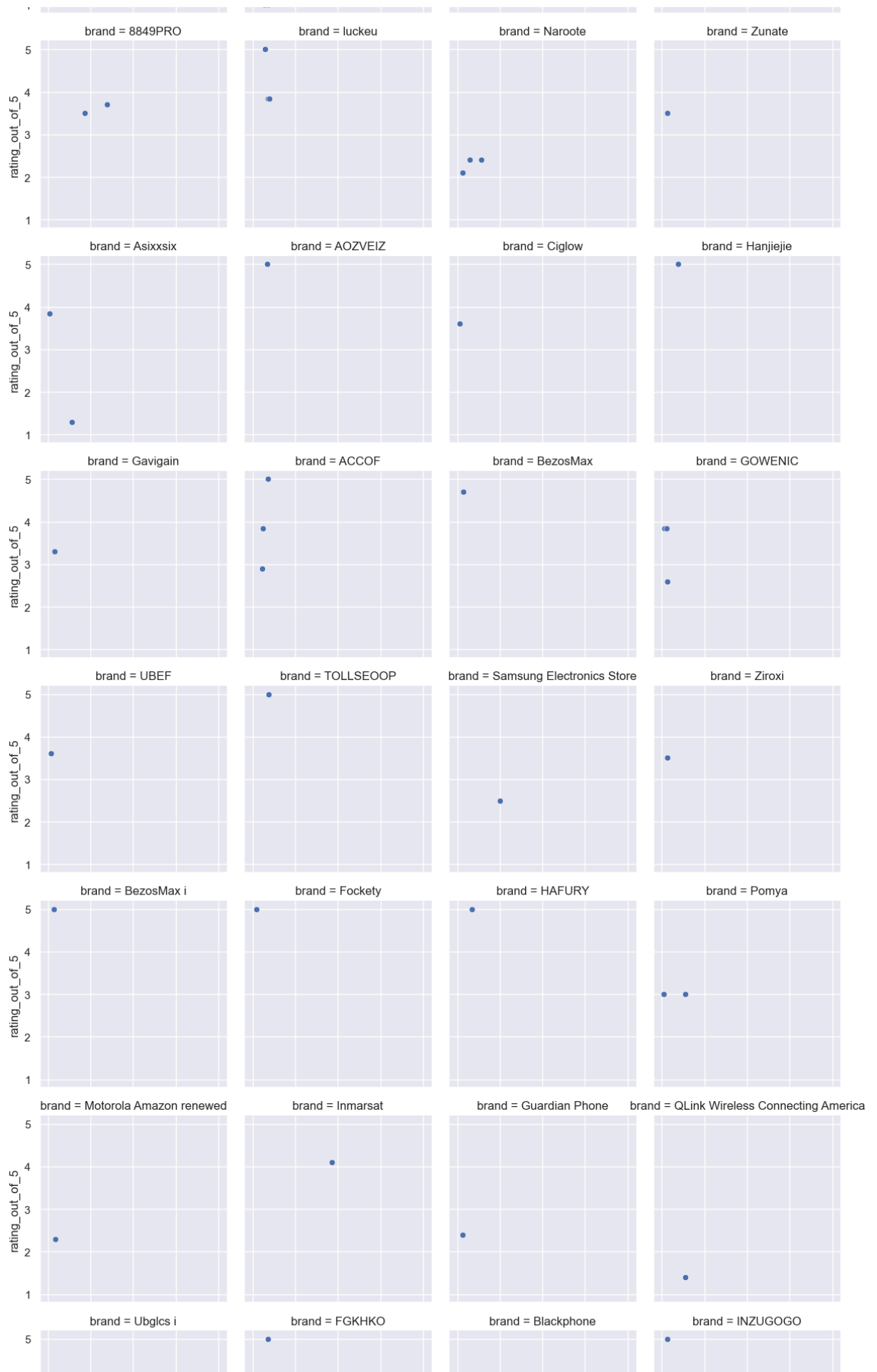




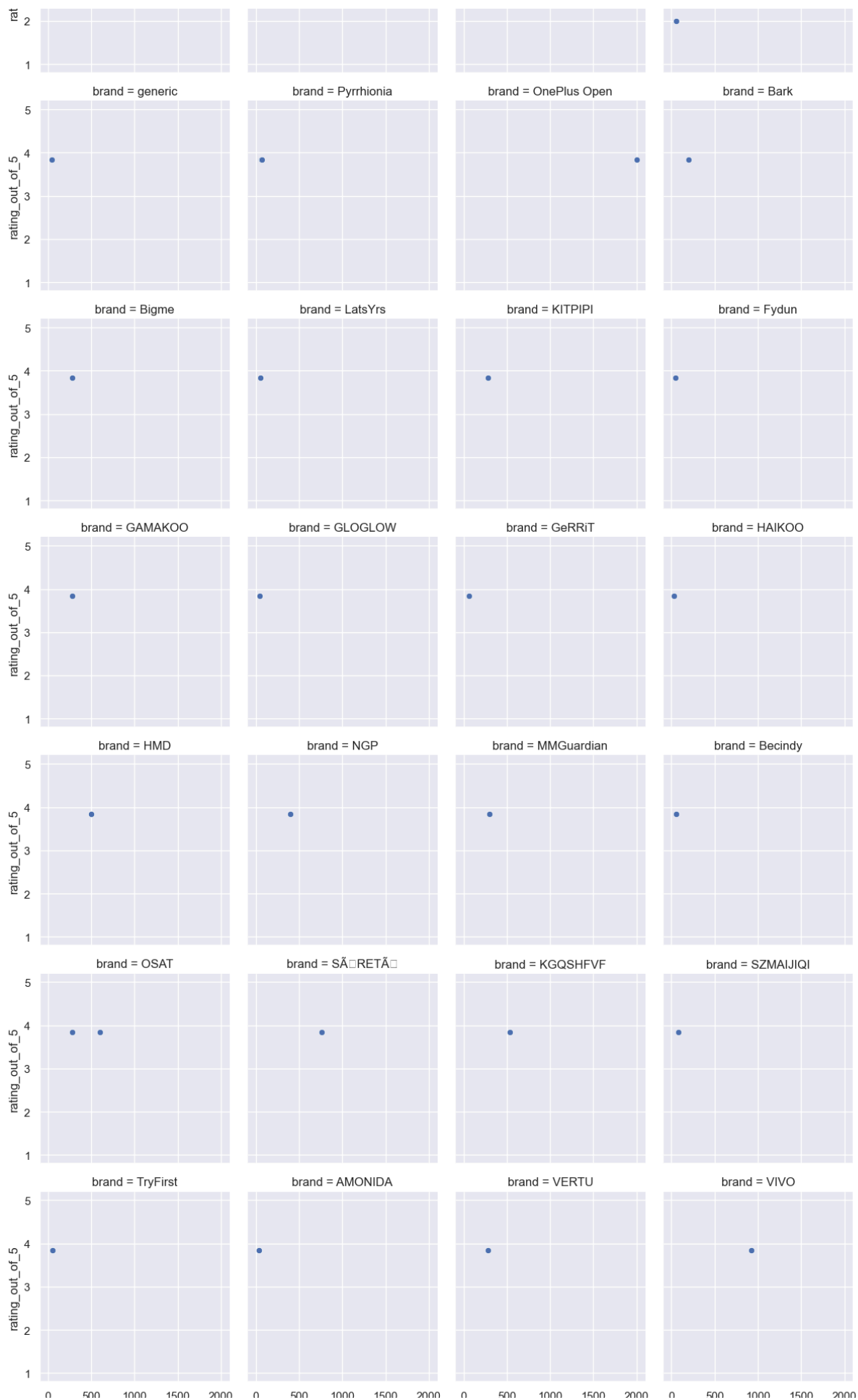






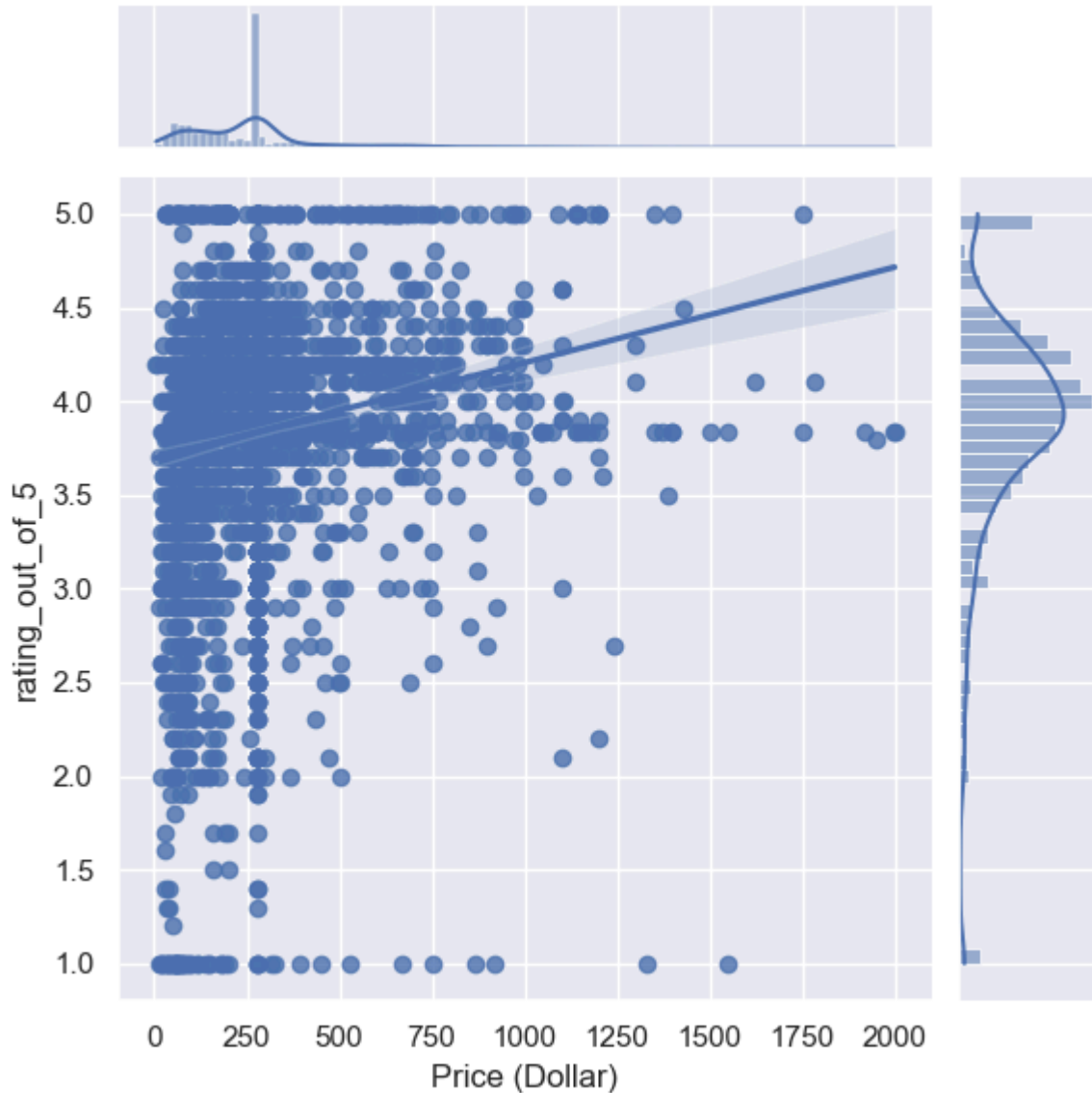






8. Price vs Rating

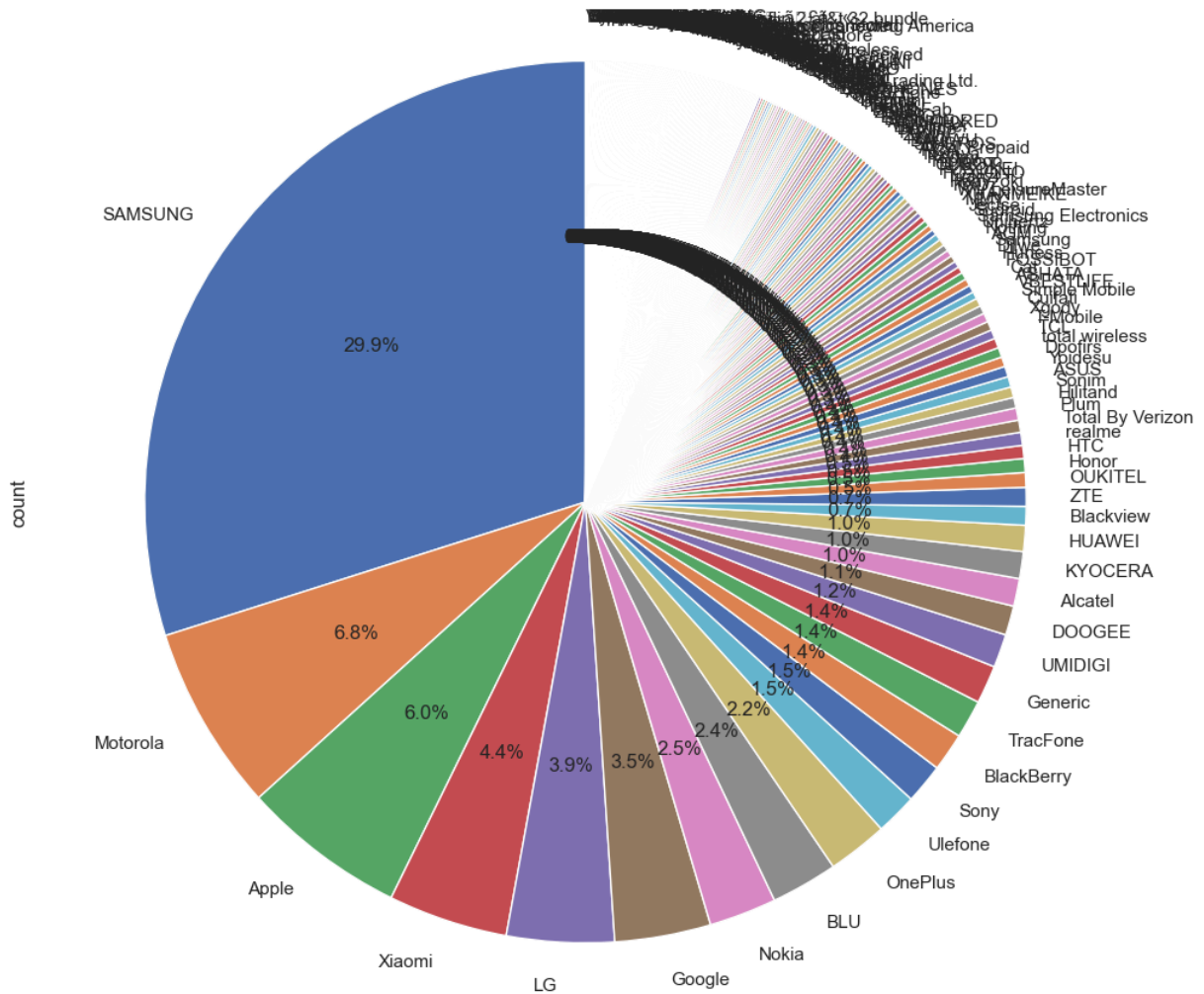
```
In [20]: sns.set(rc = {'figure.figsize' : (22, 11)})
sns.jointplot(x = "Price (Dollar)", y = "rating_out_of_5", data = data, kind = "reg")
plt.show()
```



9. Product counts by brand

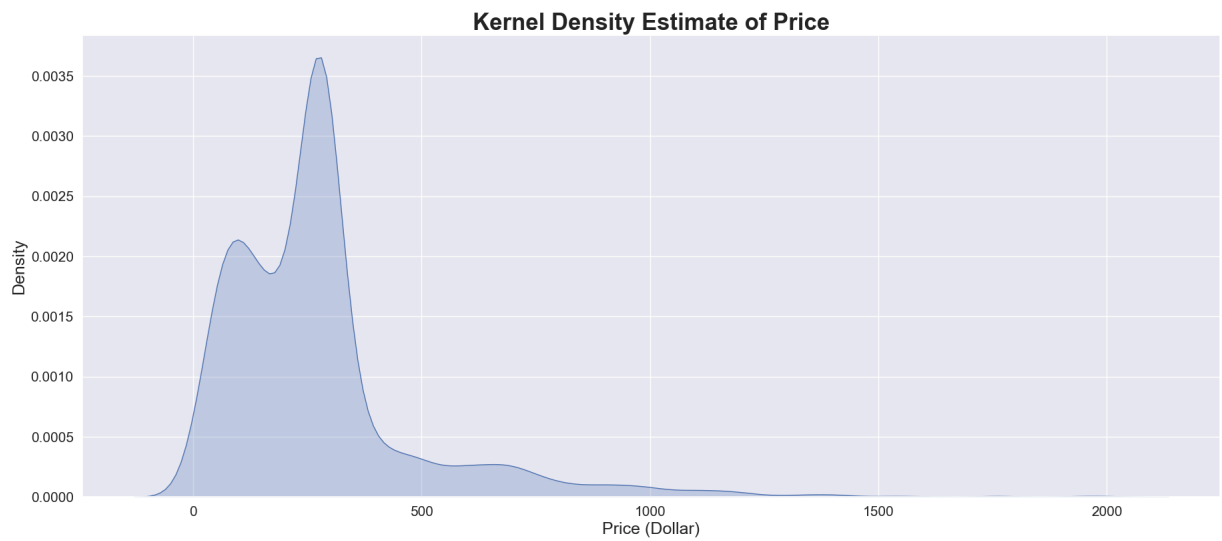
```
In [21]: plt.figure(figsize=(25, 12))
data["brand"].value_counts().plot.pie(autopct = '%1.1f%%', startangle = 90)
plt.title('Product Distribution by Brand', fontweight = "bold", fontsize = 25)
plt.show()
```

Product Distribution by Brand



9. Price (Dollar)

```
In [22]: plt.figure(figsize=(22, 9))
sns.kdeplot(data['Price (Dollar)'], shade=True)
plt.title('Kernel Density Estimate of Price', fontweight = "bold", fontsize = 25)
plt.xlabel("Price (Dollar)", fontsize = 18)
plt.ylabel("Density", fontsize = 18)
plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.show()
```



10. Correlation Heatmap

```
In [23]: sns.set(rc = {'figure.figsize' : (22, 12)})
s = data.select_dtypes(include = "number").corr()
sns.heatmap(s, annot = True)
plt.title("Correlation Heatmap", fontweight = "bold", fontsize = 35)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.show()
```

